



A Framework for Quantifiable Process Improvement through Method Fragments in Situational Method Engineering

Michael Chima Ogbuachi*

Itilekha Podder*

micogb@inf.elte.hu

itilekha19@inf.elte.hu

Eötvös Loránd University

Budapest, Hungary

Udo Bub

Eötvös Loránd University

Budapest, Hungary

udobub@inf.elte.hu

Murad Huseynli

Eötvös Loránd University

Budapest, Hungary

murad@inf.elte.hu

ABSTRACT

In the field of Business Process Management (BPM), planning and executing precise process tailoring and optimization can be done through different design strategies. Even though Situational Method Engineering (SME) can provide an additional layer of constitutional knowledge, it hasn't been explored as deeply as other traditional methods. Strategies that rely on a so-called "situational context" make use of the atomic conceptual entities known as method chunks and/or fragments from the field of Situational Method Engineering (SME). BPM, on the other hand, describes processes through representational tools that have been thoroughly used in industry and established as reliable. All of these designs have advantages and disadvantages. We analyzed several designs and proposed a synthesized framework (metamodel) that combines their strong points, while also providing a way to objectively quantify and restructure the performance of a pre-existing process/product (or optimize the creation of an entirely new one). We provide an analysis with a manufacturing organization using BPM concepts for process management/improvement and our proposed method framework, which incorporates Situational Method Engineering metamodeling and the Critical Path Method as a base for process improvement. We show here how using our proposed framework brings a flexible approach to a structured process management, helping enterprises to define, apply, store and retrieve their processes through methods/fragments, while also providing a guideline for systematic tailoring.

CCS CONCEPTS

• **Computing methodologies** → **Modeling methodologies**; *Model verification and validation*; **Systems theory**; • **Mathematics of computing** → **Information theory**; *Linear programming*; *Network optimization*; • **Social and professional topics** → *Computer supported cooperative work*; *Reengineering*; *Project management techniques*; *Systems analysis and design*.

*Both authors contributed equally to this research.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

AISS 2021, November 26–28, 2021, Sanya, China

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-8586-2/21/11...\$15.00

<https://doi.org/10.1145/3503047.3503535>

KEYWORDS

Situational Method Engineering, Method Fragments, Metamodeling, Business Process Management, Business Process Evaluation, Critical Path Method

ACM Reference Format:

Michael Chima Ogbuachi, Itilekha Podder, Udo Bub, and Murad Huseynli. 2021. A Framework for Quantifiable Process Improvement through Method Fragments in Situational Method Engineering. In *2021 3rd International Conference on Advanced Information Science and System (AISS 2021)*, November 26–28, 2021, Sanya, China. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3503047.3503535>

1 INTRODUCTION

Structured Strategy like Business Process Management (BPM) has been adopted by several organizations to analyse and continually improve their fundamental activities, such as manufacturing, marketing and so on. through problem identification, modeling, automation, analysis and performance measurement. To support the BPM at the organizational level, situation-specific method engineering has been used for more than two decades [6]. "Method engineering" is the engineering discipline of designing, constructing and adapting methods, techniques and tools for the development of information systems [4]. Situational Method Engineering (SME) is a flexible approach of software method/s creation, usage, and adaptation based on the contextual information. The "One-size-fits-all" concept imposes restrictions for BPM, which is why there was a substantial need of implementations that followed the agile methodology. Every organization and/or projects have different requirements and process flows which can not be defined using a rigid structure. This is where SME comes into the picture. SME attempts to customize the pre-existing methods to find the most suitable method for the situation [15] specific to certain processes. Moreover, often times the positive/negative yields of processes can be evaluated only in the aftermath, or (in the initial stages) only rough estimations can be given about expected results. Here, we propose a framework that allows to structure a process in a mathematically measurable manner beforehand, so that optimization can be performed statically (before the process "run-time") or dynamically (through iterative improvements).

2 BACKGROUND

The main element of SME is the "method part", which can be either a small piece of an existing methodology or a completely new method. Methods are created or modified using base templates called "classes", which contain detailed descriptions of specifics

tasks, workflows, processes and/or product descriptions. Sets of classes are usually combined together to form a “metamodel”. A “method base” contains method chunks and/or method fragments. These can be seen as small parts of a method, which are meant to be tailored by a method engineer according to the requirements of an organization or project, or according to the available technology that enables performing a process. Figure 1 depicts this high level components of SME and how they are interconnected. The knowledge stored in the method base comes from construction guidelines and the *context of use*, meaning that the decisions are taken from assessing the specific situation.

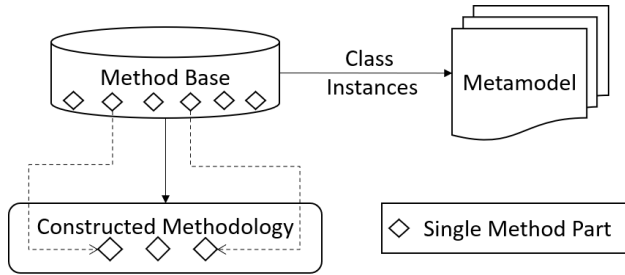


Figure 1: High level structure of SME [15]

A “Method Part” can be explained using the concept of method fragments, method chunks, and method components. Herein we are going to discuss in detail only method fragments, as those were the main focus area of our research. The method fragment is considered to be the smallest building block (or an element) of a metamodel [29]. It can be defined at any level of granularity, thus providing high flexibility and reusability [22]. In general, there are three kinds of fragments (Producer, Work unit and Work product) which are used to describe a metamodel, where the word metamodel denotes a set of rules which are closely related to an ontology [13]. In the context of process and method-focused metamodeling, a Method Fragment is created to be compatible with the respective metamodel element/class [3]. Metamodeling is used to link business processes and service composition. Through the usage of method fragments, the method rationale is expressed in terms of Guidelines, Constraints and Outcome. Method fragments are more goal specific, while the method rationale lays emphasis on the path taken to reach the goal, thus giving more emphasis on the significance of an intention. This is useful for method reconstruction, verification, consistency evaluation and method assembly. Once a method has been created using an SME approach, it is set as the base model. Later, due to reasons such as project-specific customization, some adjustments to the established base model may be required. The enactment of this customization is called “Tailoring” [9]. Process configuration is a suggested way of tailoring a method [2]. At times tailoring can lead to multiple project-specific methods from its base model; In this case an evaluation metric can be used to ensure quality. SME is used not only to create well-defined, efficient method parts, but also to maintain re-usability (which is why the documentation, typically stored in the same method base, is of primary importance).

The Method Engineering Language (MEL) [5] is a very useful tool for not only the documentation of a method fragment, but also to enact the maintenance of method fragments in a method base, by using “*methods administration functions*”. The contextual retrieval process for method fragments is especially important, but one should as well maintain the granularity of the fragments (as defined, in line with their purpose). Quality assessment of method parts (in this case method fragments) and the overall method base is crucial, to ensure the cohesiveness and completeness of the method and to check the usability of the stored components.

Linear Programming (LP) is a widely used mathematical technique for achieving the most optimal solution for a given problem, through a simplified formal representation of constraints expressed as linear relations. It has been intensively used in Business Planning, Industrial Engineering etc. [32], and its success comes from the fact that problems can be represented as mathematical models, for which the solution can be computed both by humans and machines. Nonetheless, it still requires a certain skill-set, especially when trying to model complex problems.

Here we propose a novel framework, which makes use of a “Reduced Fragment Descriptor”(RFD) for easy representation, storage and retrieval of method fragments, which helps maintaining quality and consistency within the method base. Further, we propose the usage of RFDs in the field of BPM. We incorporate the concept of “Critical Path Management”, and introduce the usage of LP for quantifiable process tailoring. We also suggest that the documentation of a method fragment, as well as that of a complete method, should include a standardized and repeatable scheme for analysis of the operability of such method. This can be obtained in several ways, both conceptual and practical.

3 RESEARCH WORK

3.1 The Reduced Fragment Descriptor

In the following paragraphs, we present our specification of a representational structure for method fragments. We call this structure *Reduced Fragment Descriptor* (RFD)(Figure 2a), and the aim is to provide, as a fundamental part of this framework, a graphical tool (or arrangement) to easily visualize the core elements that constitute a method fragment, i.e. the meaningful parts that are typically required during a business or process analysis. It is important to remember that the method fragments described throughout this paper correspond to the atomic SME entity as per the definition given in the OPF metamodel [8, 12], the ISO/IEC 24744 [18] metamodel (in particular) and summarized by Henderson et al. in [14]. In this regard, our representation is compatible with both the process and product fragment categorizations.

As visible in Figure 2, in our representation we try to include two fundamental attributes of the description of method components [8, 15, 20, 33], which we found not to be so clear in the standard method fragment templates. Namely, these are the input and output attributes. Considering that SME is conceptually very close to industrial applications, we think it is especially relevant to show them. In fact, these properties are perhaps quite relatable when considering product fragments, even though process fragments can indeed be characterized by them. In addition to that, explicitly showing this information makes sense, since method fragments

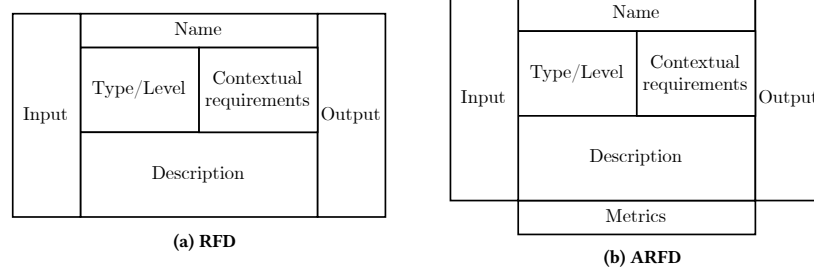


Figure 2: The two variants of our Reduced Fragment Descriptor

originated through an analogy with software components [14], similarly to elements coming from the ISO/IEC 12207 standard [17]. The six constituent parts of the RFD are described in Table 1.

Table 1: Essential Elements of a Reduced Fragment Descriptor (RFD)

Essential Elements	Basic Description
Name	The name given to the method fragment.
Type/Level	Type of action, typically identified as use, create, modify, in accordance with the ActionKind type in [18].
Contextual requirements	Definition of the scope this method fragment is intended for (context of use [15]), and the requirements of such scope.
Description	This is a concise description of what operations are involved in this method.
Input	Incoming information that this method fragment needs to operate.
Output	Information/artifacts produced by the method fragment.

Each RFD should come with its comprehensive “documentation”, i.e. a compendium of documents that essentially completes the definition of an RFD, therefore constituting the full fragment specification (which we recommend being in line with the template presented in [18]). This documentation includes an evaluation of the fragment itself, from a method engineering standpoint. We describe the tool we recommend for such evaluation in Section 3.4.

3.2 Graphical Method Composition

Cossentino et al. (2007) [7] show two base approaches for the construction of software engineering processes, starting from the retrieval of pre-existing or ad hoc method fragments (of which they also provide a revised specification) from a method base. Their work was performed for the construction of multi-agent systems. Of these two approaches, the development-process-driven strategy proposes the construction of a process/method through a sequence of phases, which should be ordered based on the chosen SEP life-cycle model [17]. Each phase is represented by one or more method fragments. This approach is arguably the one that is closest to what we are suggesting here, as it can be easily related to a process-flow representation that is based on sequential “blocks”. As [15]

points out, a process method constructed this way can be easily visualized through graphical tools, such as UML Activity Diagrams. In [26, 31], examples involving the use of Activity Diagrams for method creation can be found. As with all these strategies, our approach supposes that the method fragments are already defined and can be used after retrieving them from a method base. Thus, expertise (or experiential knowledge) on situation-aware method construction is still required here. Though in our case, by not using Activity Diagrams, we limited the required notation to the use of *Augmented RFDs* (Figure 2b) as nodes of a network diagram. An Augmented RFD (ARFD) is basically a version of the structure in Figure 2a, to which the value of one or more quantifiable metrics has been added. An example of addition of metrics to method fragments for enterprise processes has been explored by Saeki in [24]. It is important to mention that, most often, metrics can only be identified (and computed) during the actual process design phase, and this is why the two representations for RFD and ARFD are kept as separate: while the RFD represents the more conceptual method fragment, the ARFD comes as a consequence of its use.

3.3 Process Tailoring Based on Quantifiable Measures

Fragment tailoring and process tailoring are herein conceptually distinguished. The first can be perceived as in the work of Ter Hofstede and Verhoef (1997) [29], as consisting of customization and integration steps, where customization simply comprises the addition or deletion of actions associated with a method fragment [15]. We explain the broader problem of process tailoring below (seen as process improvement, on a larger scope). An ARFD is an additional blueprint, through which we want to make it easy to analyse a process while also keeping open the possibility of using tools that generally pertain to the field of BPM. Method fragments here become “nodes” in the graphical representation of a process, and the representation itself can be composed using structures such as Activity-on-Arrow (AON) network diagrams, which we particularly advise for this framework. This new extended diagram (extended in terms of the addition of SME data through RFDs) is what we called *Process Schema* (PS).

It is visible in Figure 3 how this structure facilitates visualizing when the output of a fragment is a necessary input to a subsequent fragment, and how such a dependency can in fact be carried along

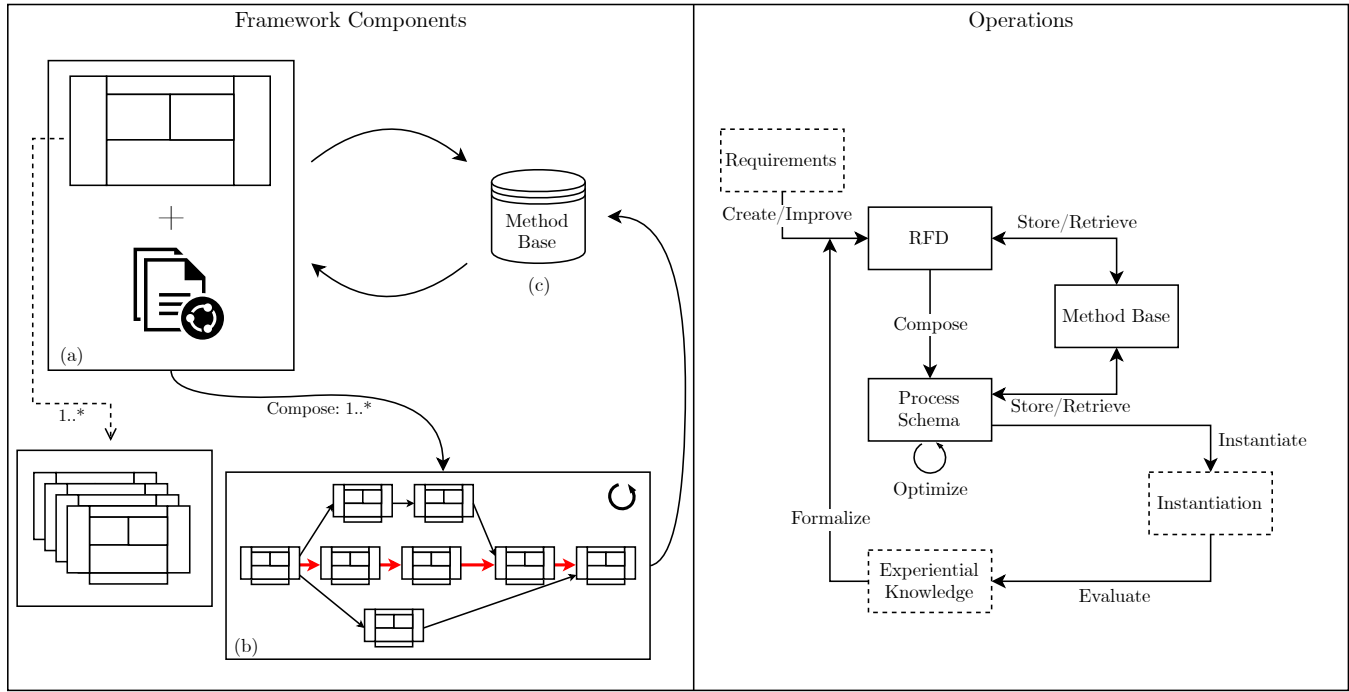


Figure 3: Our framework for quantifiable process improvement, including the relations between the RFDs and their documentation (a), the PS in which they participate (b) and the method base (c).

the flow of a process. This adds up to the idea of operational sequence that is typical of this type of diagram, in which an action can have the result of a previous action as a prerequisite [16, 27]. Though in our PS representation we “enforce” the connection of RFDs through their output and input fields, this does not imply that every RFD must produce an output (even though it is usually expected that an action would yield a result, that being in the form of a document or just a new “state” of the project). Either of the two fields can be left empty if, for instance, no output is produced or no input is required. Paths in a PS (flows in the process) can be weighed based, for instance, on a linear composition of the aforementioned metrics (which practically constitute the “generalized costs” of the nodes within a network diagram). Though we emphasize the use of network diagrams in this work, it is important to underline that other structures may successfully be used as well, e.g. Activity-on-Arrow (AOA) diagrams, such as PERT [16, 30]. The selection of network diagrams as a base for this framework is founded on the choice of keeping a minimal notation to promote ease of use. On top of that, an AON structure arguably makes it easier to identify the nodes and their weight as the independent variables of an optimization problem. Furthermore, since a measure of “cost” can be computed for a path, analytical studies such as the Critical Path Method (CPM) [30] can be carried out on it. This is a process which industry, especially in the manufacturing field, is already very acquainted with. Figure 3 also shows the operations that a user of our framework would have to perform, at various levels of process definition.

Optimizing the critical path truly is a mathematical problem with respect to method fragments as well, a problem for which techniques aside from CPM, such as Operation Research [16] and LP [1], can also be applied. Among these, the latter is the one we suggest the use of, because of the complexities that can arise during process development. In fact, processes can really be seen as constantly evolving entities, and this means that it can often be hard to identify how the constraints imposed by the operational context might influence the possibility of improving a process, even when improvement might just mean adding or removing an action (identified by the respective process fragment).

Several studies have described how the use of linear programming in conjunction with network diagrams can enable new possibilities in process/project management. An example can be found in the recent work by JR San Cristóbal Mateo [25], who devised an integer linear programming (ILP) model applied to CPM. He incorporated time, cost, quality and safety as variables in the optimization of processes of the construction industry, to avoid project failure as early as possible. Nevertheless, examples of successful combination of CMP and linear programming arose as early as 1973 with the work conducted by Swanson, Lloyd A. [27], again meant to aid project management. Another extensive analysis of such an approach can be found in Bernard W. Taylor’s work [28]. The essence of the strategy lies in the minimization of project crashing costs. Project crashing [19] is a process that involves reducing the time of completion of a project (identified by the critical path) by reducing the time taken to fulfil single activities; it solves the

problem of minimizing the additional costs, while also minimizing process duration [1, 21].

$$\begin{aligned} & \min \sum_{i=0}^n s_i x_i \quad (1) \\ & \text{s.t.} \\ & x_i \leq u_i \\ & y_{i+1} \geq y_i - x_i + d_i \\ & y_{final} \leq T \text{ (desired finishing time)} \\ & y_{final} \geq 0 \\ & x_i \geq 0; i \in \{\text{activities}\} \\ & y_i \geq 0; i \in \{\text{activities}\} \end{aligned}$$

An example of a simple LP optimization plan for process crashing is shown in Equation 1. Here, for each activity $i \in A$, s_i is the cost slope, determined as $s = \frac{\Delta \text{cost}}{\Delta \text{time}}$; x_i is the amount of reduction; y_i is the starting time (y_{i+1} is used for the starting time of the immediate following activity, and y_{final} for the starting time of the final activity); u_i is the maximum time reduction; d_i is the normal duration, and T is the desired finishing time for the project. The scope that this approach can cover is not limited to problems of time and cost. As mentioned previously, linear programming allows for flexibility in the amount of detail that the optimization problem should consider, making it suitable for the complex scenarios that project management can generally present. As an example, problems such as managing the allocation of personnel that is participating to the activities of a process can also be tackled. This type of information, while being considered as a metric, actually has its origin in the situational context. Therefore, this approach towards optimization can effectively span through different levels of detail in a process definition, while apparently just focusing on a more generalized scope of process tailoring. Thus, we can say that process tailoring indicates which steps (method fragments) need to be improved through fragment tailoring, while possibly providing detailed information on some specific aspects of the fragment that could be addressed to enact such improvement. These aspects may be represented by corresponding metrics, which are also derived from the situational context.

3.4 Method Evaluation

We previously introduced the need for an evaluation performed at each step of a process design, which should be methodical and easily repeatable by nature. Gregor and Jones (2007) [11] established a design theory meant for Information Systems Research (ISR), constituted by eight components.

These eight components, as shown in Table 2, were taken by P. Offerman, U. Bub et al. [23] and refined, characterizing an implementation specific to methods (the original components were meant to define a process for building Design Theory, in a much more generalized scope). These refinements added important operational detail to the structure of the theory, mostly in two forms: *attributes* of the component and *evaluation criteria*. Attributes outline a structural definition of the method, whereas evaluation criteria give a qualitative analysis of each DSR component. These questions were selected through a hermeneutical analysis of the components

Table 2: The Eight Components of IS Design Theory by [11], refined by [23]

Components	Intention
Purpose and Scope	Describes the “intention” of a method and characteristics of the output it produces, given a situational context.
Constructs	Properties of a method, identified from its structure, its output or the situational context. These may also derive from the justificatory knowledge of application/situation-specific approaches.
Principles of form and function	In information systems, these describe the architecture; for methods, these establish a strict relation between the elements of the method and the meta-model that defined it.
Artefact mutability	An evaluation of two kinds of mutability for methods: 1) Changes on the method itself, that the designer can predict; 2) Mutability that can still be expected (and is described in the original method design), but happens during the instantiation of the method.
Testable propositions	Analysis to have a complete view of what the statements made about the method/artefact are, that can be measured and tested.
Justificatory knowledge	Meant to clearly state all the theory supporting the method, the product, and the context in which the method is being applied. As in [10], different kinds of theory can be selected in different situations.
Principles of implementation	Helps exploring what the necessary steps are, during the implementation phase. This usually implies adapting a method to the specific scope (situation or organization) it should be used in. It is also known as the <i>tailoring process</i> .
Expository instantiation	Meant to show a real-life application of the method, either as a description of it being applied in a specific context, or as information about its execution.

and revisited several times, to ensure that they would help understanding if method modelling is being performed “correctly” and/or coherently. We consider the refined eight components of IS design theory for the evaluation of a method throughout its life-cycle: from its first definition, through the several mutations it may undergo.

4 CONCLUSION AND FUTURE WORK

In this paper, we have proposed a novel framework for quality improvement and assessment for various industrial processes (with a major focus towards Information Systems). This framework is an amalgamation of core concepts taken from different fields, such as SME, DSR, BPM, and CPM.

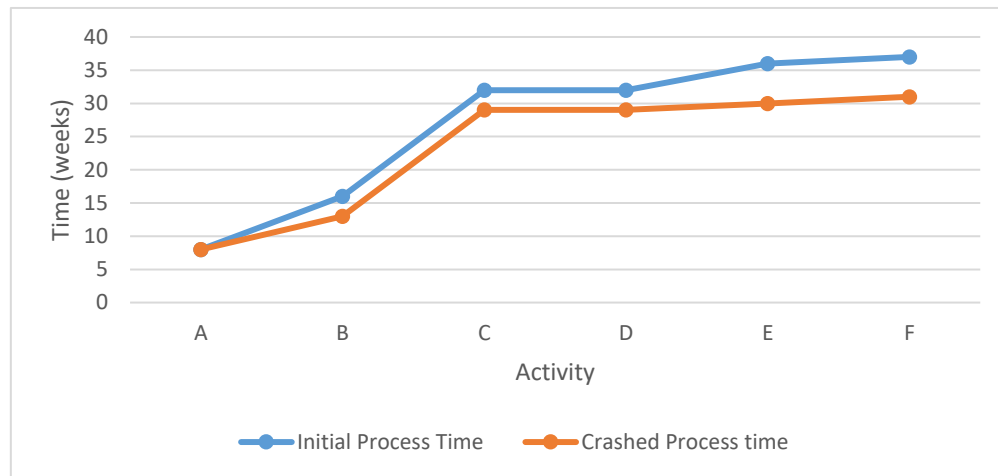


Figure 4: Cumulative time reduction to an ML-based sensor improvement project using LP (activity D does not affect the critical path).

In Figure 4, we show the result of the time reduction we achieved on a project related to sensor manufacturing process¹, by applying our framework to the original process definition. For the sake of space and simplicity, only time is shown here; the rest of the project is described in an upcoming paper. What is important to mention, though, is that through our process we managed to define a situation-aware project crashing structure. The result was an acceptable increase of the total project cost by 6.37% for the added resources, with the majorly impacted activities being B and E. Specifically, this caused the following changes in the situational context of the aforementioned activities: B) Changes related to the personnel (improving the coordination of highly skilled personnel in the activity, and integrating additional workforce for some specific sub-activities, increased the overall operational speed); E) Improvements to the technological stack, such as the inclusion of statistical testing tools to perform more thorough comparisons of different ML inference models. All this work was documented through respective method fragments, and saved as updated references in the method base designed for the company.

We have shown that our approach provides multiple levels of flexibility, allowing to model processes into reusable situational methods. This is done by using method fragments to ease addressability, storage and usage, thereby minimizing design efforts. This approach ensures that the granularity of tasks in a process remains cohesive with any further updates or extension. Using LP enriches the quality of process improvement by providing the benefits of flexible adaptation, while providing a detailed view of complex process chains. For future work, we plan to further validate this framework on different business domains such as the sensor manufacturing industry, software development industry etc. The usage of this framework as a foundation for a new *Computer Aided Method Engineering* (CAME) tool is also currently being explored.

REFERENCES

- [1] Wallace Agyei. 2015. Project Planning And Scheduling Using PERT And CPM Techniques With Linear Programming: Case Study. *International Journal of Scientific & Technology Research* 4 (2015), 222–227.
- [2] Marko Bajec, Damjan Vavpotič, and Marjan Krisper. 2007. Practice-driven approach for creating project-specific software development methods. *Information and Software Technology* 49, 4 (apr 2007), 345–365. <https://doi.org/10.1016/j.infsof.2006.05.007>
- [3] J. Bezivin and O. Gerbe. 2001. Towards a precise definition of the OMG/MDA framework. In *Proceedings 16th Annual International Conference on Automated Software Engineering (ASE 2001)*. 273–280. <https://doi.org/10.1109/ASE.2001.989813>
- [4] Sjaak Brinkkemper. 1996. Method engineering: Engineering of information systems development methods and tools. *Information & Software Technology* 38 (04 1996), 275–280. [https://doi.org/10.1016/0950-5849\(95\)01059-9](https://doi.org/10.1016/0950-5849(95)01059-9)
- [5] Sjaak Brinkkemper, Motoshi Saeki, and Frank Harmsen. 2001. A Method Engineering Language for the Description of Systems Development Methods. 473–476. <https://doi.org/10.1007/3-540-45341-5-33>
- [6] Tobias Bucher and Robert Winter. 2010. *Taxonomy of Business Process Management Approaches*. Springer Berlin Heidelberg, Berlin, Heidelberg, 93–114. <https://doi.org/10.1007/978-3-642-01982-1-5>
- [7] Massimo Cossentino, Salvatore Gaglio, Alfredo Garro, and Valeria Seidita. 2007. Method fragments for agent design methodologies: from standardisation to research. *IJAOSSE* 1 (01 2007), 91–121. <https://doi.org/10.1504/IJAOSSE.2007.013266>
- [8] D.G. Firesmith and B. Henderson-Sellers. 2002. *The OPEN Process Framework: An Introduction*. Addison-Wesley. <https://books.google.hu/books?id=rA6gSeTnnngC>
- [9] Brian Fitzgerald, Nancy L. Russo, and Tom O’Kane. 2003. Software Development Method Tailoring at Motorola. *Commun. ACM* 46, 4 (April 2003), 64–70. <https://doi.org/10.1145/641205.641206>
- [10] Shirley Gregor. 2009. Building theory in the sciences of the artificial. In *Proceedings of the 4th International Conference on Design Science Research in Information Systems and Technology - DESRIST ’09*. ACM Press. <https://doi.org/10.1145/1555619.1555625>
- [11] Shirley Gregor and David Jones. 2007. The Anatomy of a Design Theory. *Journal of the Association for Information Systems* 8, 19 (05 2007), 312–. <https://doi.org/10.17705/1jais.00129>
- [12] Brian Henderson-Sellers. 2002. Process Metamodelling and Process Construction: Examples Using the OPEN Process Framework (OPF). *Annals of Software Engineering* 14, 1/4 (2002), 341–362. <https://doi.org/10.1023/a:1020570027891>
- [13] Brian Henderson-Sellers, Tony Clark, and Cesar Gonzalez-Perez. 2013. On the Search for a Level-Agnostic Modelling Language. 240–255. <https://doi.org/10.1007/978-3-642-38709-8-16>
- [14] Brian Henderson-Sellers, Cesar Gonzalez-Perez, and Jolita Ralyte. 2008. Comparison of Method Chunks and Method Fragments for Situational Method Engineering. *Proceedings of the Australian Software Engineering Conference, ASWEC*, 479–488. <https://doi.org/10.1109/ASWEC.2008.4483237>

¹The names of the project and company are not disclosed for the blind review

- [15] Brian Henderson-Sellers, Jolita Ralyte, Pär Ågerfalk, and Matti Rossi. 2014. *Situational Method Engineering*. <https://doi.org/10.1007/978-3-642-41467-1>
- [16] F.S. Hillier and G.J. Lieberman. 2002. *Introduction to Operations Research*. McGraw-Hill Higher Education. <https://books.google.hu/books?id=xH9dNAEACAAJ>
- [17] International Organization for Standardization. 2002. *ISO/IEC 12207:1995/Amd 2:2002 Information technology – Software life cycle processes, institution = International Organization for Standardization*. Technical Report.
- [18] ISO/IEC 24744:2014. 2014. *Software engineering – Metamodel for development methodologies*. Standard. International Organization for Standardization/International Electrotechnical Commission, Geneva, CH.
- [19] Arun Kanda and U.R.K. Rao. 1984. A network flow procedure for project crashing with penalty nodes. *European Journal of Operational Research* 16, 2 (May 1984), 174–182. [https://doi.org/10.1016/0377-2217\(84\)90072-9](https://doi.org/10.1016/0377-2217(84)90072-9)
- [20] Fredrik Karlsson and Pär Ågerfalk. 2009. Towards Structured Flexibility in Information Systems Development: Devising a Method for Method Configuration. *Journal of Database Management* 20 (07 2009), 51–75.
- [21] Chitra Karmaker and Pobitra Halder. 2017. Scheduling Project Crashing Time Using Linear Programming Approach: Case Study. *International Journal of Research in Industrial Engineering (In press)* (11 2017).
- [22] Karel Lemmen, Fred Mulder, and Sjaak Brinkkemper. 1999. *Education and Information Technologies* 4, 2 (1999), 181–202. <https://doi.org/10.1023/a:1009604118404>
- [23] Philipp Offermann, Sören Blom, Olga Levina, and Udo Bub. 2010. Proposal for Components of Method Design Theories. *Business & Information Systems Engineering* 2, 5 (Aug. 2010), 295–304. <https://doi.org/10.1007/s12599-010-0120-x>
- [24] M. Saeki. 2003. Embedding Metrics into Information Systems Development Methods: An Application of Method Engineering Technique. In *CAiSE*. https://doi.org/10.1007/3-540-45017-3_26
- [25] J. R. San Cristóbal Mateo. 2019. An Integer Linear Programming Model Including Time, Cost, Quality, and Safety. *IEEE Access* 7 (2019), 168307–168315. <https://doi.org/10.1109/ACCESS.2019.2953185>
- [26] V. Seidita, J. Ralyté, B. Henderson-Sellers, M. Cossentino, and Nicolas Arni-Bloch. 2007. A comparison of deontic matrices, maps and activity diagrams for the construction of situational methods. In *CAiSE Forum*.
- [27] Lloyd A. Swanson. 1973. Linear programming: an approach to critical path management. *Project Management Quarterly* 4, 1 (04 1973), 14–21.
- [28] Bernard W. Taylor. 2010. *Introduction to management science* (10th ed ed.). Prentice Hall, Upper Saddle River, N.J. OCLC: ocn295000560.
- [29] Arthur H.M. ter Hofstede and T.F. Verhoef. 1997. On the feasibility of situational method engineering. *Information Systems* 22, 6 (1997), 401–422. [https://doi.org/10.1016/S0306-4379\(97\)00024-0](https://doi.org/10.1016/S0306-4379(97)00024-0)
- [30] A. Toal, T.V. Sobczak, M. Toal, and Australian Executive Development Foundation. 1967. *P.E.R.T./C.P.M.: Programme Evaluation Review Technique, Critical Path Method*. Australian Executive Development Foundation. <https://books.google.hu/books?id=QxULAQAAMAAJ>
- [31] Inge van de Weerd, Sjaak Brinkkemper, Jurriaan Souer, and Johan Versendaal. 2006. A situational implementation method for web-based content management system-applications: method engineering and validation in practice. *Software Process: Improvement and Practice* 11, 5 (2006), 521–538. <https://doi.org/10.1002/spip.294> arXiv:<https://onlinelibrary.wiley.com/doi/pdf/10.1002/spip.294>
- [32] Robert J. Vanderbei. 2020. *Linear Programming*. Springer International Publishing. <https://doi.org/10.1007/978-3-030-39415-8>
- [33] Kai Wistrand and Fredrik Karlsson. 2004. Method Components – Rationale Revealed, Vol. 3084. 189–201. https://doi.org/10.1007/978-3-540-25975-6_15